

Multiple Inheritance in Java vs. Python

What Is Multiple Inheritance?

Multiple inheritance allows a class to inherit attributes and methods from more than one parent class. This can lead to complexities, especially when different parent classes have methods with the same name, resulting in ambiguity known as the **diamond problem**.

Multiple Inheritance in Python

Python's Support for Multiple Inheritance

Python supports multiple inheritance directly. A class can inherit from multiple parent classes, allowing it to access all their attributes and methods. ([Reddit](#))

Example: Multiple Inheritance in Python

```
class Animal:
    def speak(self):
        print("Animal speaks")

class Mammal(Animal):
    def speak(self):
        print("Mammal speaks")

class Bird(Animal):
    def speak(self):
        print("Bird chirps")

class Bat(Mammal, Bird):
    pass

bat = Bat()
bat.speak()
```

Output:

```
Mammal speaks
```

In this example, `Bat` inherits from both `Mammal` and `Bird`. When `speak()` is called, Python uses the **Method Resolution Order (MRO)** to determine which method to execute. ([Programiz](#))

Method Resolution Order (MRO)

Python uses the **C3 linearization algorithm** to determine the MRO. This ensures a consistent and predictable order in which base classes are searched when executing a method.

You can view the MRO of a class using:

```
print(Bat.__mro__)
```

Output:

```
(<class '__main__.Bat'>, <class '__main__.Mammal'>, <class  
'__main__.Bird'>, <class '__main__.Animal'>, <class 'object'>)
```

The Diamond Problem in Python

The diamond problem occurs when a class inherits from two classes that both inherit from a common superclass. This can lead to ambiguity in method resolution.

Python handles this gracefully using MRO. Here's an example:([LinkedIn](#))

```
class A:  
    def greet(self):  
        print("Hello from A")  
  
class B(A):  
    def greet(self):  
        print("Hello from B")  
  
class C(A):  
    def greet(self):  
        print("Hello from C")  
  
class D(B, C):  
    pass  
  
d = D()  
d.greet()
```

Output:([Medium](#))

```
Hello from B
```

In this case, D inherits from both B and C, which in turn inherit from A. When `greet()` is called on an instance of D, Python follows the MRO and executes B's version of `greet()`.



Multiple Inheritance in Java

✗ Java's Approach to Multiple Inheritance

Java does **not** support multiple inheritance with classes to avoid the diamond problem. However, it allows a class to implement multiple interfaces, providing a form of multiple inheritance.([Oracle Documentation](#))



Example: Implementing Multiple Interfaces in Java

```
interface Flyable {
```

```

        void fly();
    }

    interface Swimmable {
        void swim();
    }

    class Duck implements Flyable, Swimmable {
        public void fly() {
            System.out.println("Duck flies");
        }

        public void swim() {
            System.out.println("Duck swims");
        }
    }

    public class Main {
        public static void main(String[] args) {
            Duck duck = new Duck();
            duck.fly();
            duck.swim();
        }
    }

```

Output:

```

Duck flies
Duck swims

```

In this example, `Duck` implements both `Flyable` and `Swimmable` interfaces, providing concrete implementations for their methods.

🧩 Handling the Diamond Problem with Interfaces

Starting from Java 8, interfaces can have default methods with implementations. If a class implements multiple interfaces that have default methods with the same signature, the class must override the method to resolve the ambiguity.

```

interface A {
    default void greet() {
        System.out.println("Hello from A");
    }
}

interface B {
    default void greet() {
        System.out.println("Hello from B");
    }
}

class C implements A, B {
    public void greet() {
        // Resolving the ambiguity
        A.super.greet();
    }
}

```

```
public class Main {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.greet();  
    }  
}
```

Output:

Hello from A

Here, class C implements both interfaces A and B, each having a default `greet()` method. To resolve the conflict, C overrides `greet()` and explicitly calls A's version using `A.super.greet();`.

Key Differences Between Java and Python

Feature	Python	Java
Multiple Class Inheritance	✅ Supported	❌ Not Supported
Multiple Interface Inheritance	✅ Supported	✅ Supported
Method Resolution	C3 Linearization (MRO)	Explicit override required when conflicts arise
Diamond Problem Handling	Resolved via MRO	Avoided by disallowing multiple class inheritance
Flexibility	High	Moderate
Complexity	Higher due to potential ambiguities	Lower due to restrictions